# Information retrieval: the lexical foundation

SciFact

LECTURE 19 LECTURE NOTES

---

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

# Part V begins

| Part | Lectures | Source |
|---|---|---|
| I–IV · the textbook | 1–18 | Géron, Chapters 1–15 |
| **V · Search and recommendation** | **19–22** | **these lecture notes** |
| VI · Multimodal models | 23–24 | Géron, Chapters 15–16 |

**OUTSIDE THE BOOK, AND EXAMINABLE**

Four lectures the textbook does not cover, because search and recommendation are the two applications of machine learning most of you will actually meet, and because they are where the embeddings of Part IV earn their living. They are examined on exactly the same terms as everything else.

# Where we are

Eighteen lectures of *prediction*: given an input, produce a label, a number or a sequence.

> **RETRIEVAL IS NOT THAT**
>
> The input is a **query** and a **corpus**, and the output is a **ranking** of the corpus. There is no fixed set of classes — the corpus changes daily. There is no single right answer — several documents may be relevant, and their order matters. And you cannot score the whole corpus with a neural network for every query, because there are millions of documents and the user is waiting.

So the metrics are different, the baselines are different, and the first method is not a model at all.

# Today

1. The retrieval problem, and the corpus (10 min)
2. **The method** — the inverted index, term weighting, and BM25 (35 min)
3. **The mathematics** — evaluating a ranking: MRR, AP and NDCG (30 min)
4. Where the judgements come from, and what pooling misses (15 min)

# What you should be able to do afterwards

- State why retrieval is not classification, in terms of what the output is
- Build an inverted index, and say what it costs and what it saves
- Write BM25 down, and say what each of its three parts corrects for
- Compute P@k, R@k, RR, AP and NDCG@k by hand from a relevance pattern
- Choose between them from the question being asked, and defend the choice
- Say what a reported retrieval score does *not* tell you

The paper asks the fourth and fifth of these more often than any other content in Part V.

FIRST TEN MINUTES

# The problem, and the corpus

10 minutes

# The task

## A **claim** arrives. Find the scientific abstracts that bear on it.

- “0-dimensional biomaterials lack inductive properties.”
- “Breast cancer development is determined exclusively by genetic factors.”

Nobody wants a yes or no. They want the *evidence*, ranked, so a human can read the top few and decide. That is the whole shape of the problem, and it is why the output is an ordering.

> **WHY THIS CORPUS**
>
> SciFact is small enough that we can score **every** document for **every** query exactly, with no approximate index in the way. Every number in this lecture and the next is therefore a property of the method, not of an index we did not examine.

# Two questions that look alike

| | Classification | Retrieval |
|---|---|---|
| Input | one document | a query *and* a corpus |
| Output | a label from a fixed set | an ordering of the corpus |
| Classes | fixed, known at training time | the corpus, changing daily |
| Right answer | one | several, and their order matters |
| Cost per prediction | one forward pass | the whole corpus, if you are naive |

You could phrase retrieval as classification — one class per document — and it would be useless: the classes change every time a document is added, and there are millions of them with a handful of examples each.

# The shape of every search system

### 1 Retrieve

millions → hundreds. Must be cheap.

### 2 Re-rank

hundreds → tens. Can be expensive.

### 3 Present

tens → ten. Diversity, freshness, policy.

Today is stage 1, and it is stage 1 that decides the ceiling: **a document the first stage never returns cannot be recovered by any later stage**. Stage 2 is Lecture 20. Stage 3 is mostly not machine learning, and mostly not in this course.

This is why we will keep looking at recall@100 for a system whose output is ten results. The first stage is not judged on what the user sees; it is judged on what it makes possible.

# Attention falls off a results page

Whatever we build, its output is read by a person, and that person's attention is not uniform:

- The first result is examined far more often than the second
- Almost nobody reaches the tenth
- The second page effectively does not exist

> **CONSEQUENCE FOR THE MATHEMATICS**
>
> A metric that treats ranks 1 and 10 alike is **measuring something nobody experiences**. Every metric in this lecture's derivation is an attempt to encode that decay — and the differences between them are differences of opinion about its shape.

Nothing in this lecture measures user attention: it is an assumption we import, and I am flagging it as one.

# What we will not use

- **No search library.** The index is a Python dictionary built in the notebook; BM25 is a sum you can read
- **No approximate index.** Every document is scored for every query, exactly — so a number is a property of the method, never of a recall/latency setting we did not inspect
- **No training.** Nothing in this lecture has a parameter fitted to data. That is next lecture's subject, and the comparison only means something if today's baseline is honest

In production you would use none of these choices. They are here so that nothing between you and the arithmetic is opaque.

# The corpus, counted

| Quantity | SciFact |
|---|---|
| Abstracts | 5,183 |
| Claims with judgements (test) | 300 |
| Mean abstract length, in tokens | 225.2 |
| Median abstract length | 213 |
| Vocabulary | 35,734 |
| Relevant documents per claim, mean | 1.13 |
| Claims with exactly one relevant document | 92.3% |

That last row matters more than it looks. With one relevant document per claim most of the time, **recall@10 and the reciprocal rank are measuring nearly the same thing** — and on a corpus where each query had twenty relevant documents they would not be.

# What a document looks like

**Microstructural development of human newborn cerebral white matter assessed in vivo by diffusion tensor magnetic resonance imaging.**

> Alterations of the architecture of cerebral white matter in the developing human brain can affect cortical development and result in functional disabilities. A line scan diffusion-weighted magnetic resonance imaging (MRI) sequence with diffusion tensor analysis was applied to measure the apparent diffusion coefficient, to calculate …

311 tokens of title and abstract, and nothing else — no full text, no citations, no authors. Whatever a system knows about this paper, it knows from this.

Notice how it is written: “diffusion-weighted magnetic resonance imaging”, not “MRI scan”. A claim written by someone else about the same result may share very few of these words, and that observation will return, with a number attached, before this lecture ends.

# The baseline: return the corpus unranked

Rule 2 of this course, in a new place. Before any method, what does *no* method score? Return every document in corpus order, the same order for every query:

| Metric | Fixed order |
|---|---|
| Recall@10 | ✗ 0.3% |
| Reciprocal rank | ✗ 0.0017 |
| NDCG@10 | ✗ 0.0012 |

Effectively zero, and that is the point: unlike accuracy under imbalance in Lecture 3, a ranking metric has an honest floor near zero. **Any** number you report is above the baseline, so “better than nothing” is not an argument here — the comparison that matters is against the *previous method*.

THE METHOD

# The inverted index, and BM25

35 minutes · examinable

# First: what is a term?

Before any weighting, a decision nobody writes up: how the text becomes a list of things to count. On a real claim from our corpus:

> “0-dimensional biomaterials show inductive properties.”

`0` `dimensional` `biomaterials` `show` `inductive` `properties`

6 tokens, lower-cased, split on non-letters. Which is already three decisions — and `0-dimensional` has just become `0` and `dimensional`, two terms that mean nothing apart.

# Tokenisation decides what can be found

| Term | Documents containing it |
|---|---|
| `0` | 951 |
| `dimensional` | 72 |
| `biomaterials` | 1 |
| `show` | 1,012 |
| `inductive` | 2 |
| `properties` | 211 |

`biomaterials` occurs in **1** abstract of 5,183, and `show` in 1,012. One of those terms decides the answer; the other decides nothing.

✗ **Note what has already happened: a query term appearing in one document makes the task nearly trivial, and a term appearing in a fifth of the corpus makes it nearly impossible. The weighting scheme has not been chosen yet, and the outcome is already largely determined by the tokeniser.**

# What we are not doing: stemming

Our tokeniser treats these as unrelated terms:

| | | | |
|---|---|---|---|
| `cell` 1,837 | `cells` 1,969 | `cellular` 514 | |
| `infect` 10 | `infects` 7 | `infection` 337 | `infected` 166 |

A stemmer would merge each row, pooling the evidence: a query for `infect` would then match the 337 abstracts containing `infection` instead of only 10.

It also merges things that should not be merged, and the net effect on scores is corpus-dependent and usually small. We leave it out because **every unexplained step between the text and the number is somewhere a result can hide** — and because the mismatch it half-solves has a better answer next lecture.

# The obvious first idea: Boolean retrieval

Return the documents that contain *all* the query terms. It is exact, it is fast with an inverted index, and it was how search worked for thirty years.

$$\text{result} = \bigcap_{t \in q} \text{postings}(t)$$

It also needs no notion of a score, which is either elegant or fatal. Before reading on: on a corpus of scientific abstracts and claims written by scientists, how many abstracts do you expect to contain *every* word of a claim?

# Boolean retrieval, measured

| Over all 300 test claims | |
|---|---|
| Claims matching **no** document at all | **✗ 98.0%** |
| Mean documents matching every term | 0.023 |
| Most any claim matched | 2 |

**✗ 98.0% of claims return an empty page. Not a bad ranking — nothing at all.**

The conjunction is brittle: one word of the claim absent from an otherwise perfect abstract, and the abstract is excluded. And the failure is silent, because an empty result set looks like “no such document exists”.

> **THE LESSON THAT REORGANISES EVERYTHING**
>
> Stop **filtering** and start **scoring**. Every document gets a number, the user gets the largest, and a missing term costs something rather than everything.

# Why retrieval is cheap

Scoring 5,183 documents per query by reading each one is hopeless at scale. The trick is 50 years old and it is a data structure, not a model.

> **THE INVERTED INDEX**
>
> For every *term*, store the list of documents containing it, with the count. A query of four words then touches only the documents containing at least one of those four — not the corpus.

| | |
|---|---|
| Vocabulary (distinct terms) | 35,734 |
| Postings (term–document pairs) | 633,514 |
| Full matrix, were it dense | 185 million cells |

The postings list is **0.3%** of the dense matrix. Sparsity is not an optimisation here; it is the reason the whole field is possible.

# Term frequency, and why it is not enough

A document mentioning “biomaterials” five times is probably more about biomaterials than one mentioning it once. So score by how often the query terms occur.

> **TWO FAILURES, IMMEDIATELY**
>
> **Common words dominate.** Every abstract contains “of” and “the”, so a query containing them matches everything equally.
>
> **Long documents win.** A 900-word abstract contains more of everything than a 100-word one, and is not thereby more relevant.

The rest of the method is two corrections, one for each failure — and each has a parameter, so each can be measured.

# Scoring: the cost problem, stated

Scoring every document sounds worse than filtering, not better. Naively, per query:

| | |
|---|---|
| Documents to touch | 5,183 |
| Tokens to read | 1.17 million |

And this is a *tiny* corpus. A web index is $10^{10}$ documents, and the user is waiting 200 ms.

But almost every one of those documents shares **no term at all** with the query, and contributes exactly zero to any sensible score. Reading them is the waste.

# Building the index

One pass over the corpus, counting:

```
post = defaultdict(list)
for i, doc in enumerate(docs):
    for term, tf in Counter(doc).items():
        post[term].append((i, tf))
```

Four lines, and it is the entire data structure the field is built on. Scoring then visits only $\bigcup_{t \in q} \text{postings}(t)$ — the documents that can possibly score above zero:

```
s = np.zeros(N)
for t in query:
    for i, tf in post.get(t, []):
        s[i] += weight(t, tf, len[i])
```

The weighting function is the only thing left to decide, and the next four slides decide it.

# Saturation, and the parameter $k_1$

The first correction to raw counting: the tenth occurrence of a word should not count ten times the first.

$$\frac{\mathrm{tf}\,(k_1+1)}{\mathrm{tf}+k_1} \longrightarrow k_1+1 \ \text{ as } \mathrm{tf}\to\infty$$

| tf | 1 | 2 | 3 | 5 | 10 | 20 |
|---|---|---|---|---|---|---|
| factor | 1.000 | 1.310 | 1.462 | 1.610 | 1.743 | 1.818 |

From 1 to 2 occurrences the factor rises 0.310; from 10 to 20 it rises 0.075. The curve is bounded, so no single word can dominate a score by repetition — which is also, incidentally, the first anti-spam device in search.

# Choosing $k_1$ by measurement

$k_1$ sets how fast the saturation bites: small $k_1$ saturates almost immediately, large $k_1$ approaches raw counting. Measured over 150 claims, $b$ held at 0.4:

| $k_1$ | 0.5 | 0.9 | 1.2 | 2.0 |
|---|---|---|---|---|
| NDCG@10 | 0.6883 | 0.6937 | 0.6944 | 0.6926 |

The spread across a fourfold change in $k_1$ is 0.0061. Which is the honest finding, and not the one you would guess from how much has been written about tuning it: **on abstracts, this parameter barely matters**.

It matters on long documents, where a term can occur fifty times. Report the corpus with the parameter, always.

# Inverse document frequency

$$\mathrm{idf}(t) = \log\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right)$$

$N$ DOCUMENTS, $N_T$ OF THEM CONTAINING $T$

| Term | Documents | idf |
|---|---|---|
| `of` | 5,173 | 0.0020 |
| `the` | 5,159 | 0.0047 |
| `and` | 5,158 | 0.0049 |

`of` appears in 5,173 of 5,183 abstracts and earns a weight of 0.0020. **No stop-word list was needed**: the weighting derives the stop words from the corpus it is given.

# Stop words, derived rather than declared

Classical systems shipped a hand-written list of words to delete: *the, of, and, in, to* … It works, and it is a liability:

- The list is language-specific, and someone must maintain it
- It is corpus-blind — in a corpus about the band The Who, deleting *the* and *who* destroys the query
- The decision is **binary**: a word is deleted or kept, with nothing in between

> **IDF REPLACES THE LIST**
>
> A weight of 0.0020 is not deletion, it is near-deletion — and it is *computed from this corpus*. If a word is common here, it is discounted here. Nobody maintains anything.

This is the same move as learning a representation instead of engineering features, arriving thirty years earlier.

# Length normalisation, and the parameter $b$

$$\mathrm{tf}' = \frac{\mathrm{tf}}{\mathrm{tf} + k_1 \left(1 - b + b \, \frac{|d|}{\mathrm{avgdl}}\right)}$$

- $b = 0$ — no normalisation; long documents keep their advantage
- $b = 1$ — length divided out completely

| $b$ | 0.0 | 0.4 | 0.75 | 1.0 |
|---|---|---|---|---|
| NDCG@10 | 0.6861 | 0.6937 | 0.6917 | 0.7011 |

The spread is small, and that is the finding: SciFact abstracts are of similar length (225 tokens on average), so there is little for this correction to correct. On web pages there would be.

# BM25, assembled

$$\mathrm{BM25}(q,d) \;=\; \sum_{t \in q} \mathrm{idf}(t)\; \frac{\mathrm{tf}_{t,d}\,(k_1+1)}{\mathrm{tf}_{t,d} + k_1\left(1 - b + b\,\dfrac{|d|}{\mathrm{avgdl}}\right)}$$

- The **idf** factor: rare terms count for more
- The **saturating tf**: the tenth occurrence adds far less than the second, because $k_1$ bounds the growth
- The **length term**: $b$ decides how much of the document's length is divided out

**✗ Every piece is there because a specific failure demanded it. That is why BM25 has survived thirty years of replacement attempts, and it is the baseline every neural retriever is measured against — including the one in Lecture 20.**

# One document, scored term by term

The claim “Incidence rates of cervical cancer have decreased.” against the abstract BM25 ranked first (274 tokens):

| Term | tf | idf | contribution |
|---|---|---|---|
| **`cervical`** | **3** | **4.736** | **6.786** |
| **`incidence`** | **5** | **3.238** | **5.145** |
| `rates` | 2 | 2.979 | 3.802 |
| `cancer` | 5 | 1.902 | 3.023 |
| `have` | 3 | 1.008 | 1.445 |
| `of` | 6 | 0.002 | 0.003 |
| **Total** | | | **20.204** |

`cervical` occurs 3 times and is worth 6.786. `of` occurs 6 times — twice as often — and is worth 0.003.

That ratio is the whole method working. And this abstract **is** one of the judged-relevant ones.

# BM25, measured

| Metric | Fixed order | BM25 |
|---|---|---|
| Precision@1 | 0.0000 | **0.5433** ✓ |
| Recall@10 | 0.0033 | **0.7784** ✓ |
| Recall@100 | 0.0117 | **0.8852** ✓ |
| MRR | 0.0017 | **0.6350** ✓ |
| NDCG@10 | 0.0012 | **0.6611** ✓ |

The right abstract is in the top ten for **77.8%** of claims, and top of the list for **54.3%**. No training, no labels, no gradient — a data structure and two corrections.

Worth holding on to: **recall@100 is 88.5%**. Almost everything relevant is retrieved *somewhere* in the first hundred. That gap between @10 and @100 is exactly what re-ranking exists to close, in the next lecture.

# Which correction was doing the work?

Three corrections were argued for. Rule 3 — every model needs an ablation — because an argument is not a measurement. Each removed in turn, full test set:

| Scoring function | NDCG@10 |
|---|---|
| raw term frequency | 0.0944 |
| no idf | 0.5350 |
| no saturation, no length | 0.4801 |
| no length normalisation | 0.6490 |
| **BM25** | **0.6611** |

Counting raw words scores 0.0944 — barely above returning the corpus in its stored order.

# What the ablation actually says

- **idf is most of the method.** Removing it costs 0.1261; without it, common words drown the rare ones that identify the document
- **Saturation is the one we did not isolate.** Dropping saturation and length together costs 0.1809, and length alone costs 0.0120 — so most of that gap is saturation, but an experiment that removes two things at once cannot say how much
- **Length normalisation is nearly free here**, worth 0.0120 — consistent with the $b$ sweep, and with abstracts being of similar length

> **THE TRANSFERABLE PART**
>
> Not the ordering — that is a fact about SciFact. It is that **each term of a formula can be removed and priced**, and that a component nobody has priced is a component nobody has justified. The same procedure applies to every architecture in Part IV.

# Where we are, halfway

- Retrieval returns an ordering, not a label — and the first stage sets the ceiling for everything after it
- Filtering fails: 98.0% of claims match no document under conjunction
- Scoring works, and the index makes it cheap
- BM25 = idf × saturating tf × length normalisation, scoring **0.6611** with no training

Two things remain. First, the honest limit of what we have built — because it has one, and it is severe. Then the mathematics of how any of these numbers were computed at all.

# The failure BM25 cannot fix

"In mice, P. chabaudi parasites are able to proliferate faster early in infection when inoculated at lower numbers than when inoculated at high numbers."

This claim has exactly one relevant abstract in the corpus. BM25 ranks it ✗ **4,999** of 5,183 — the worst placement of any single-evidence claim in the test set.

| | |
|---|---|
| Query terms in the vocabulary | 19 |
| Terms the relevant abstract contains | ✗ **2** |
| Which terms | `in`, `to` |
| Mean idf of the shared terms | 0.0378 |
| Mean idf of the missing terms | 3.2928 |

**✗ The abstract shares `in` and `to` with the claim it is evidence for, and those carry an idf of 0.0378. There is nothing for BM25 to match on. No parameter setting rescues this.**

# Vocabulary mismatch, in general

That was the worst case. It is not an isolated one — over every judged claim–abstract pair in the test set:

| | |
|---|---|
| Mean fraction of query terms *absent* from the relevant abstract | 40.9% |
| Pairs sharing fewer than half the query terms | 28.6% |

A relevant document is missing **40.9%** of the query's words on average. People restate, abbreviate, use the Latin name, or describe the mechanism instead of naming it — and a method whose only evidence is shared strings cannot follow them.

**THE STRUCTURAL LIMIT**

BM25 matches **strings**. Relevance is about **meaning**. That gap is not a tuning problem, and it is exactly what an embedding is for — which is Lecture 20, and why Part V comes after Part IV rather than before it.

# The ceiling, and what is left on the table

| Metric | BM25 | The most it could be |
|---|---|---|
| Recall@10 | 0.7784 | 1.0000 |
| Recall@100 | 0.8852 | 1.0000 |
| NDCG@10 | 0.6611 | 1.0000 |

Two different opportunities, and they need different methods. Between recall@10 (77.8%) and recall@100 (88.5%) sit documents BM25 *found* but ranked too low — a **re-ranking** problem, solvable by an expensive model on a hundred candidates.

The remaining 11.5% were never retrieved at all. No re-ranker can reach them; only a first stage that matches on something other than words can. Those are the two halves of Lecture 20.

DERIVATION 19 · EXAMINABLE

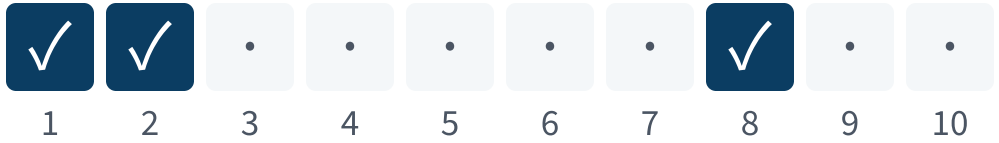
# Evaluating a ranking

30 minutes

# The example we will use throughout

## “Incidence rates of cervical cancer have decreased.”

3 abstracts in the corpus are relevant to this claim. Here is where BM25 actually put them, in its top ten:




Relevant at ranks **1, 2, 8**.

**HOW THIS EXAMPLE WAS CHOSEN**

It is the first test claim for which BM25 puts at least three relevant abstracts in the top ten. Chosen so the arithmetic is *visible*: a single hit gives a one-term sum and demonstrates nothing about a discount. It is therefore better than typical, and the honest summary of the method is the mean over all 300 claims — the table you saw, not this slide.

# Step 0 · What we are given

A derivation needs its inputs named. For one query:

- A **ranked list** $d_1, d_2, \ldots, d_n$ — the system's output, an ordering of the corpus
- A set $R$ of **relevant documents** — the judgements, made by people, treated as truth for now

Everything that follows is a function of those two objects and nothing else. No scores, no probabilities, no model — which is why the same metrics apply to BM25 today and to a transformer next lecture.

> **THE ONE ASSUMPTION**
>
> That $R$ is correct and complete. It is neither, and the last section of this lecture is about what that costs.

# Step 1 · Precision and recall, at a cutoff

- $\mathrm{P@}k = \dfrac{|\{\text{relevant}\} \cap \text{top-}k|}{k}$

  of what I showed you, how much was good
- $\mathrm{R@}k = \dfrac{|\{\text{relevant}\} \cap \text{top-}k|}{|\{\text{relevant}\}|}$

  of what was good, how much I showed you

On our example, with $k = 10$: all 3 relevant abstracts are inside the top ten, so $\mathrm{R@10} = \frac{3}{3} = 1$ — **perfect recall**.

**✗ And that is the problem. The third abstract is at rank 8, where a user may never look, and recall@10 cannot say so. It counts a set; it does not read an order.**

# Step 2 · The cutoff is arbitrary

Every fixed $k$ hides the same thing in a different place:

- $k$ too small — a method that fills ranks 11–20 with the answer scores zero, identically to one that never finds it at all
- $k$ too large — rank 1 and rank 100 count the same, and the metric stops rewarding the ordering it is supposed to measure

> **WHAT WE ACTUALLY WANT**
>
> A number that **decreases smoothly as a relevant document moves down**, rather than falling off a cliff at one chosen position. Every metric that follows is a different answer to that one requirement.

# Step 3 · Reciprocal rank: where is the first one?

$$\mathrm{RR} = \frac{1}{\text{rank of the first relevant document}}$$

Our example finds one at rank 1, so $\mathrm{RR} = 1/1 = 1.0000$. Averaged over queries this is **MRR**, and over all 300 claims BM25 scores 0.6350.

| First hit at rank | 1 | 2 | 5 | 10 |
|---|---|---|---|---|
| RR | 1.0000 | 0.5000 | 0.2000 | 0.1000 |

Smooth, bounded, and it moves when the order moves. But it stops reading after the first hit — our ranking would score 1.0000 whether the other two abstracts were at ranks 2 and 8 or nowhere at all.

Which makes MRR right for one kind of question — “is there a good answer near the top?” — and wrong for “did I find the evidence?”

# Step 4 · Average precision: read every hit

$$\mathrm{AP} = \frac{1}{|R|} \sum_{i\,:\,d_i \in R} \mathrm{P@}i$$

THE PRECISION AT EACH RANK WHERE A RELEVANT DOCUMENT APPEARS

| Hit at rank $i$ | Relevant so far | P@$i$ |
|---|---|---|
| 1 | 1 | 1.0000 |
| 2 | 2 | 1.0000 |
| 8 | 3 | 0.3750 |

$\mathrm{AP} = \frac{1}{3}\,(1.0000 + 1.0000 + 0.3750) = 0.7917$ — every hit contributes, and one further down contributes less, because the precision above it is worse.

# Step 5 · Where AP stops

AP assumes relevance is **binary**: a document is relevant or it is not. Often it is not that simple.

- An abstract that directly tests the claim, and one that mentions it in passing, are both “relevant” — and AP cannot prefer the first
- Web search judgements are routinely graded: perfect, excellent, good, fair, bad

> **TWO REQUIREMENTS NOW**
>
> A **gain** that depends on *how* relevant a document is, and a **discount** that depends on *where* it sits. Separating those two is the whole idea of the last metric.

# Interlude · Why a logarithm, and not something else

The discount could be any decreasing function. Three candidates, with what each implies:

| Discount | Rank 1 vs 2 | Rank 9 vs 10 | Says |
|---|---|---|---|
| $1$ (none) | equal | equal | position is irrelevant |
| $1/i$ | halves | −10% | position is nearly everything |
| $1/\log_2(i+1)$ | −37% | −4% | position matters, mildly |

No derivation picks one; they encode different beliefs about a user. The logarithm is convention, chosen because it is gentle enough that positions 5 and 10 remain distinguishable while rank 1 still leads.

**✗ Convention is not truth. If you report NDCG, you have adopted this curve, and you should be able to say so out loud.**

# Step 6 · Gain, discounted

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g_i}{\log_2(i+1)}$$

Two independent parts: a **gain** $g_i$ for how relevant the document is, and a **discount** for how far down it sits. Separating them is the idea — and the $+1$ is what keeps rank 1 from dividing by zero, since $\log_2(1) = 0$.

| Hit at rank $i$ | $\log_2(i+1)$ | contribution |
|---:|---:|---:|
| 1 | 1.0000 | 1.0000 |
| 2 | 1.5850 | 0.6309 |
| 8 | 3.1699 | 0.3155 |
| **DCG@10** | — | **1.9464** |

# Step 7 · The number is not yet comparable

DCG@10 $= 1.9464$. Is that good? The question has no answer, because DCG depends on **how many relevant documents exist** — a claim with ten relevant abstracts can reach a far larger DCG than one with three, however well both are ranked.

So divide by the best that this query could possibly have scored: the **ideal** ranking, all relevant documents first.

| Ideal rank | contribution |
|---:|---:|
| 1 | 1.0000 |
| 2 | 0.6309 |
| 3 | 0.5000 |
| **IDCG@10** | **2.1309** |

3 relevant documents, so the ideal puts them at ranks 1–3, and no ranking of this corpus can beat 2.1309.

# Step 8 · NDCG, and what it saw

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k} = \frac{1.9464}{2.1309} = 0.9134$$

| Same ranking, four metrics | Value |
|---|---|
| Recall@10 | ✗ 1.0000 |
| RR | 1.0000 |
| AP | 0.7917 |
| **NDCG@10** | **0.9134** |

**✗ Recall calls this ranking perfect. It is not: one abstract sits at rank 8.** The metric you report decides what you are allowed to notice.

# A metric that cannot read the order

**FAILURE CONDITION**

Recall@10 called this ranking perfect while a relevant abstract sat at rank 8. A set-valued metric counts what came back and never reads the order, so the number is correct and the ranking it praises may be one no user would accept.

# Interlude · One relevant document, moved

The simplest possible ranking: one relevant document, and the only question is where it sits.

| Rank | 1 | 2 | 5 | 10 |
|---|---|---|---|---|
| RR | 1.0000 | 0.5000 | 0.2000 | 0.1000 |
| NDCG@10 | 1.0000 | 0.6309 | 0.3869 | 0.2891 |
| Recall@10 | 1.0000 | 1.0000 | 1.0000 | 1.0000 |

Recall@10 is constant at 1 across the whole row: it cannot distinguish the best possible ranking from the worst one that still makes the page. RR falls fastest; NDCG falls more gently, by construction.

With 92.3% of our claims having exactly one relevant abstract, this row *is* our corpus, most of the time.

# Step 9 · Which one to report

| Question | Metric |
|---|---|
| Is there a good answer at the top? | MRR |
| Did I find everything, order aside? | Recall@k |
| Did I find everything, order counted? | AP |
| Graded relevance, position counted? | NDCG@k |
| Will the second stage have anything to work with? | Recall@100 |

**THE RULE**

Choose the metric from the **question**, before you see any scores. Choosing it afterwards, from the numbers, is how a paper reports recall@100 for the system that lost on NDCG@10 — and it is the single commonest dishonesty in this literature.

# Step 10 · Averaging over queries

One more choice, and it is usually made silently. Given per-query scores, do you average the **scores**, or pool the **counts**?

- **Macro** — mean of the per-query metric. Every query counts equally, including one with a single relevant document
- **Micro** — pool hits and totals across queries first. Queries with many relevant documents dominate

Every number in this lecture is **macro**: the mean over 300 claims of that claim's own metric. MRR and NDCG are essentially always reported that way; precision and recall sometimes are not, and the two can differ substantially.

The variance across queries is large and almost never reported. A mean NDCG of 0.6611 contains claims scoring 1 and claims scoring 0.

# Step 11 · What none of these metrics measure

- **Latency.** A ranking that takes a minute has a real score of zero, and no metric here notices
- **Redundancy.** Ten identical relevant documents score perfectly and serve the user once
- **The unjudged.** Anything the pool never saw is counted as irrelevant, which is the next section
- **Harm.** A confidently ranked wrong answer and a merely absent one score identically

> **RULE 4, IN THIS SETTING**
>
> Every one of these is a real requirement that the reported number is silent about. A metric is a *choice of what to look at*, and it is therefore also a choice of what to ignore.

LAST FIFTEEN MINUTES

# Where the judgements come from

15 minutes

# Every number so far assumed a set of relevant documents

NDCG, AP, MRR — each takes $\{\text{relevant}\}$ as given. Somebody had to decide, for 300 claims against 5,183 abstracts, which pairs are relevant.

Judging all of them is **1.55 million** decisions for this small corpus. For a web collection it is not large, it is impossible.

> **SO NOBODY DOES**
>
> Judgements are made on a **pool**: run several systems, take the top $k$ from each, judge the union, and treat everything unjudged as **not relevant**. That last clause is an assumption, and it is doing a great deal of work.

# What pooling misses

- A document no pooled system ranked highly is **unjudged**, and therefore counted as irrelevant — whether or not it is
- So a genuinely new method, which finds documents the pool never saw, is **penalised for succeeding**
- And the pool was built from the systems of its day, which means an old collection quietly encodes the assumptions of old methods

> **THIS IS NOT A FOOTNOTE**
>
> Reported gains on a mature benchmark are partly gains at *matching the pool*. It is why a method can win on the benchmark and disappoint in production — and why a leaderboard difference of a point or two is rarely worth defending.

The same shape as the label noise of Lecture 3 and the survivorship of Lecture 2: the ground truth is a measurement, made by someone, under constraints.

# SciFact specifically

- Judgements come from the authors of the claims, who cited the abstracts — so they are *unusually* reliable, and unusually **sparse**
- 92.3% of claims have exactly one relevant abstract, mean 1.13
- An abstract that supports a claim but was not cited is unjudged, and scored as a miss

**✗ Which means our 0.6611 is a lower bound on what a reader would call the quality of this ranking, not an estimate of it. Reporting it as "BM25 achieves 0.6611" without that sentence is the kind of claim this course exists to stop.**

# Graded relevance changes the answer

SciFact judgements are binary. Where they are graded, the gain in DCG is usually taken as $g_i = 2^{\mathrm{rel}_i} - 1$ rather than $\mathrm{rel}_i$ itself.

| Grade | Linear gain | $2^{\mathrm{rel}} - 1$ |
|---|---|---|
| 0 — bad | 0 | 0 |
| 1 — fair | 1 | 1 |
| 2 — good | 2 | 3 |
| 3 — excellent | 3 | 7 |

The exponential says an excellent document at rank 3 is worth more than two fair ones at ranks 1 and 2 — a claim about users, not about mathematics, and one you should be prepared to defend when you make it.

Two systems can trade places when the gain function changes and nothing else does. State which you used.

# Five questions to ask of any retrieval result

1. **Which metric, and at what cutoff?** “Better retrieval” with no metric named is not a claim
2. **Against which baseline?** If BM25 is absent from the table, ask why — it is free, and it is often close
3. **Whose judgements, produced how?** Pooled from which systems, and how long ago?
4. **Same corpus, same queries, same preprocessing?** A different tokeniser is a different experiment
5. **How large is the difference against the spread across queries?** A gain of 0.005 on 300 queries is not a finding

These are the questions of Lecture 2 and Lecture 3, restated for a task where the ground truth is itself a sample.

# Where students lose marks on this material

- Computing DCG but forgetting to divide by the ideal — and then comparing across queries
- Using $\log_2(i)$ rather than $\log_2(i+1)$, which divides by zero at rank 1
- Dividing AP by the number of *retrieved* hits instead of $|R|$ — that gives mean precision, not average precision
- Quoting a metric with no cutoff: "NDCG of 0.66" is incomplete
- Describing BM25 as "tf–idf" — it is tf–idf with the two corrections that make it work

The written paper has asked for a hand-computed NDCG from a relevance pattern in each of the last three years. Section A, four or five marks.

# Where this sits in the course

| | |
|---|---|
| Lecture 2 | a baseline before a method — here, the corpus unranked |
| Lecture 3 | a metric answers a question — here, five of them do |
| Lecture 3 | labels are made by people — here, by pooling |
| Lecture 8 | sparsity as structure — here, it is what makes retrieval possible |
| Lectures 17–18 | embeddings — next lecture, the answer to vocabulary mismatch |

Nothing in this lecture is new machinery. It is the measurement discipline of Part I applied to a task whose output is an order, plus one data structure from 1975.

# Reading

- **These lecture notes are the primary source** — lecture-19.pdf, the extended notes for this lecture. The textbook does not cover this material
- Manning, Raghavan and Schütze, *Introduction to Information Retrieval* — chapters 1, 2 and 6 for the index and term weighting, chapter 8 for evaluation. Free online
- Robertson and Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond* — where BM25 comes from, if you want the derivation we skipped
- The SciFact and BEIR papers, for how these judgements were made

Examinable content is what is on these slides and in the notebook. The reading is for depth, not for scope.

# The notebook for this lecture

**Open Lecture 19 in Colab**

- **Runs on free CPU**, in a couple of minutes. No model is trained — there is nothing here to train
- The inverted index built by hand from a dictionary, and BM25 scored exactly against every abstract
- All four metrics implemented from the derivation, and checked against the worked example on this slide deck

> **WHAT TO CHANGE**
>
> Set $b = 0$ to switch length normalisation off, and re-score. Then remove the idf factor entirely and see which of the two corrections your corpus actually needed.

# Vocabulary

| | |
|---|---|
| **Inverted index** | term → the documents containing it, with counts |
| **Postings list** | one term's entry in that index |
| **idf** | a term's weight, falling as more documents contain it |
| **BM25** | saturating tf, length-normalised, idf-weighted |
| **MRR** | mean of $1/(\text{rank of first hit})$ |
| **NDCG@k** | discounted gain over the best possible discounted gain |
| **qrels** | the relevance judgements themselves |
| **Pooling** | judging the union of several systems' top results |

# Scope

- the retrieval problem and why it is not classification; the inverted index and why sparsity makes retrieval possible; idf, saturating tf and length normalisation, and BM25 assembled from them; **Derivation 19** in full — P@k, R@k, RR, AP, DCG, NDCG and which question each answers; pooling and what unjudged documents do to a reported score — EXAMINABLE
- index compression, skip pointers, query parsing, and any particular search library — NOT EXAMINABLE — ENGINEERING
- the probabilistic derivation of BM25 from the RSJ model, query expansion and relevance feedback, learning to rank — BEYOND THE SYLLABUS

Part V is examined from these lecture notes on exactly the same terms as the textbook parts.

# Summary

1. Retrieval returns an **ordering of a corpus**, not a label; the metrics, baselines and methods all follow from that
2. The inverted index makes it cheap: postings are 0.3% of the dense matrix
3. BM25 is three corrections to counting words, each answering a stated failure, and it scores **0.6611** NDCG@10 with no training at all
4. A ranking metric answers a **question**. Recall@10 called our example perfect while an abstract sat at rank 8
5. Every one of those numbers rests on judgements someone made on a pool, with everything unjudged counted as irrelevant

# One line to keep

> **THE LINE**
>
> **A ranking is scored by a question, and the question must be chosen before the scores are seen.**

BM25 is thirty years old, needs no labels and no gradient, and reaches 0.6611 on this corpus. Any method you meet next — including every neural retriever of the next lecture — is measured against it, on the same judgements, with the metric named in advance.

# Before the next lecture

- Run the notebook, and turn off each of BM25's two corrections in turn. One of them barely matters on this corpus — be able to say which, and why
- Note the gap we leave open: recall@10 is 77.8% but recall@100 is **88.5%**. Almost everything relevant is found *somewhere* in the first hundred
- Next lecture is about the two ways to close that gap: retrieving by meaning rather than by word, and re-ranking what a cheap first stage returned

NOT PRESENTED — FOR AFTERWARDS

# Exercises Lecture 19

five, in the style of the written paper

# Exercises — Lecture 19

**1** A ranking has relevant documents at ranks 1, 3 and 10, with three relevant documents in total. Compute AP and NDCG@10, showing the sums. [6]

**2** State why $\log_2(i+1)$ is used rather than $\log_2(i)$ in DCG, and whether the choice of discount is derived or conventional. [4]

**3** BM25 has three parts. Name the failure of raw term counting each one answers, then say which part the ablation prices on its own as costliest, and which part it cannot price alone. [5]

Solutions on Lecture 20’s deck.

# Exercises — Lecture 19 (continued)

**4** 98% of claims match no document under Boolean conjunction. State the property of the conjunction that causes it, and what the field does instead. [4]

**5** Relevance judgements are made by pooling. State what happens to a document no pooled system ranked highly, and the consequence for a genuinely new method. [4]

Solutions on Lecture 20’s deck.

THE FIVE SET AT THE END OF LECTURE 18

# Solutions Lecture 18

not part of this lecture

# Solutions — Lecture 18

**1.** Scaled dot-product attention divides by $\sqrt{d_k}$. State what the variance of the unscaled dot product is,…

**✓ $\mathrm{Var}(s) = d_k$, so $s$ has standard deviation $\sqrt{d_k}$; without the division the softmax saturates and the gradient vanishes.**

- A sum of $d_k$ independent zero-mean products, each of variance 1, has variance $d_k$ — at $d_k = 64$ typical scores are already of order 8
- Large logits make the distribution nearly one-hot, so almost no gradient flows

**2.** A decoder must not attend to positions after the one it is predicting. State the mechanism that enforces it,…

**✓ A mask applied to the attention scores before the softmax, setting future positions to $-\infty$. Without it the model sees the token it is being asked to predict.**

- Attention is all-to-all by construction, so nothing in the mechanism itself respects order
- The training loss would fall and the model would be useless at generation time, when the future genuinely does not exist

# Solutions — Lecture 18 (2)

**3.** A pretrained body is fine-tuned at $10^{-3}$, the rate that worked for the from-scratch model. Predict what…

## ✓ The loss rises and does not recover — steps that size destroy what was pretrained. The lecture fine-tunes at $2 \times 10^{-5}$.

- A pretrained block already encodes something; a step tuned for random initialisation is far too large for it
- Fifty times smaller than the $10^{-3}$ that trained the model from scratch — a step tuned for random initialisation is far too large for a block that already encodes something

**4.** A vectoriser is fitted before the split on a 400-document corpus. The leaky vocabulary is measurably larger…

## ✓ Nothing measurable: 0.30 points against a seed spread of 2.53%, with the leak ahead on 10 of 20 seeds. A null result.

- A term occurring only in test documents has an all-zero column in the training rows, so logistic regression gives it coefficient exactly zero — there is no gradient to move it
- What actually leaks is the inverse document frequencies, and an idf is an average over the corpus: the leak changes the scale of features the model already had, and a regularised linear model is nearly indifferent to that

# Solutions — Lecture 18 (3)

**5.** A corpus contains duplicate documents across the train/test split. State why no pipeline fixes it, and name…

## ✓ Nothing was fitted wrongly — the rows were separated wrongly. Use a grouped split.

- `fit` and `transform` were used correctly throughout
- The repair is to keep both copies of an entry on the same side

LECTURE 20 · LECTURE NOTES

# Information retrieval: dense retrieval

Vocabulary mismatch, and why an inverted index cannot solve it
Bi-encoders, in-batch negatives, and why the negatives are most of the work
Cross-encoders and re-ranking — why the accurate model cannot go first
All of it against today's BM25, on the same judgements

Part V · Lecture notes · examinable